ACADEMIC SIMULATION PROJECT PROPOSAL

Miami Dade College AI & Robotics Lab Digital Twin

Multi-Method Simulation (DES + ABM) with Reinforcement Learning (Q-Learning vs. PPO) & Interactive 3D Visualization

INSTITUTION
Miami Dade College

COURSE
Simulation for Applied AI

TEAM MEMBERS
Jorge, Jordan, Illia, Ricardo

STATUS
Ready for Group Review

1. Executive Summary & Problem Context

> **Real-World Problem Setting:** Miami Dade College AI & Robotics Lab

In the Miami Dade College AI & Robotics Lab, physical equipment, shared component bins (Station A), soldering/prototyping workbenches (Station B), and specialized testing benches (Station C) experience acute bottlenecks during class labs and peak project hours. Autonomous Mobile Robots (AMRs) tasked with delivering parts, clearing workstations, and assisting students encounter conflicting priorities, queue congestion, and battery depletion.

Core Objectives:

- **Minimize Student Wait Times:** Cut transit and queue delays for requested lab components and bench testing slots.
- **Optimize AMR Fleet Productivity:** Maintain high throughput while guaranteeing robots recharge (Station D) before critical battery failure.
- **Benchmark AI Policies:** Compare heuristic baselines (FIFO / threshold) against **Q-Learning** and **Proximal Policy Optimization (PPO)**.

- **Interactive Stakeholder Decision Support:** Deliver insights via an interactive **Streamlit Dashboard** and an **AnyLogic** `.alp` model featuring full 3D animation.

2. Team Roles & Work Breakdown Structure (WBS)

To ensure balanced accountability and fulfill all professor peer-evaluation requirements, tasks are divided among the 4 team members:

Jorge A Mendoza

PROJECT MANAGER & ANYLOGIC MODEL ARCHITECT

- Build the AnyLogic `.alp` simulation model
- Implement DES process flow for student work orders
- Define ABM statecharts for AMRs and lab stations
- Configure the AnyLogic native 3D animation canvas

Jordan Gonzalez

REINFORCEMENT LEARNING ENGINEER

- Build Python Gymnasium lab queue environment
- Implement and train **Q-Learning** policy
- Implement and train **PPO** policy (Stable-Baselines3/PyTorch)
- Export policy models, convergence metrics, and loss logs

Illia Shybanov

DASHBOARD & INTEGRATION SPECIALIST

- Develop multi-tab **Streamlit Dashboard** (`app.py`)
- Implement interactive sliders and user parameter controls
- Visualize KPI charts (Q-Learning vs. PPO vs. Baseline)
- Embed 3D Digital Twin visual preview and live telemetry

Ricardo Molina Gonzalez

EXPERIMENTS & REPORTING LEAD

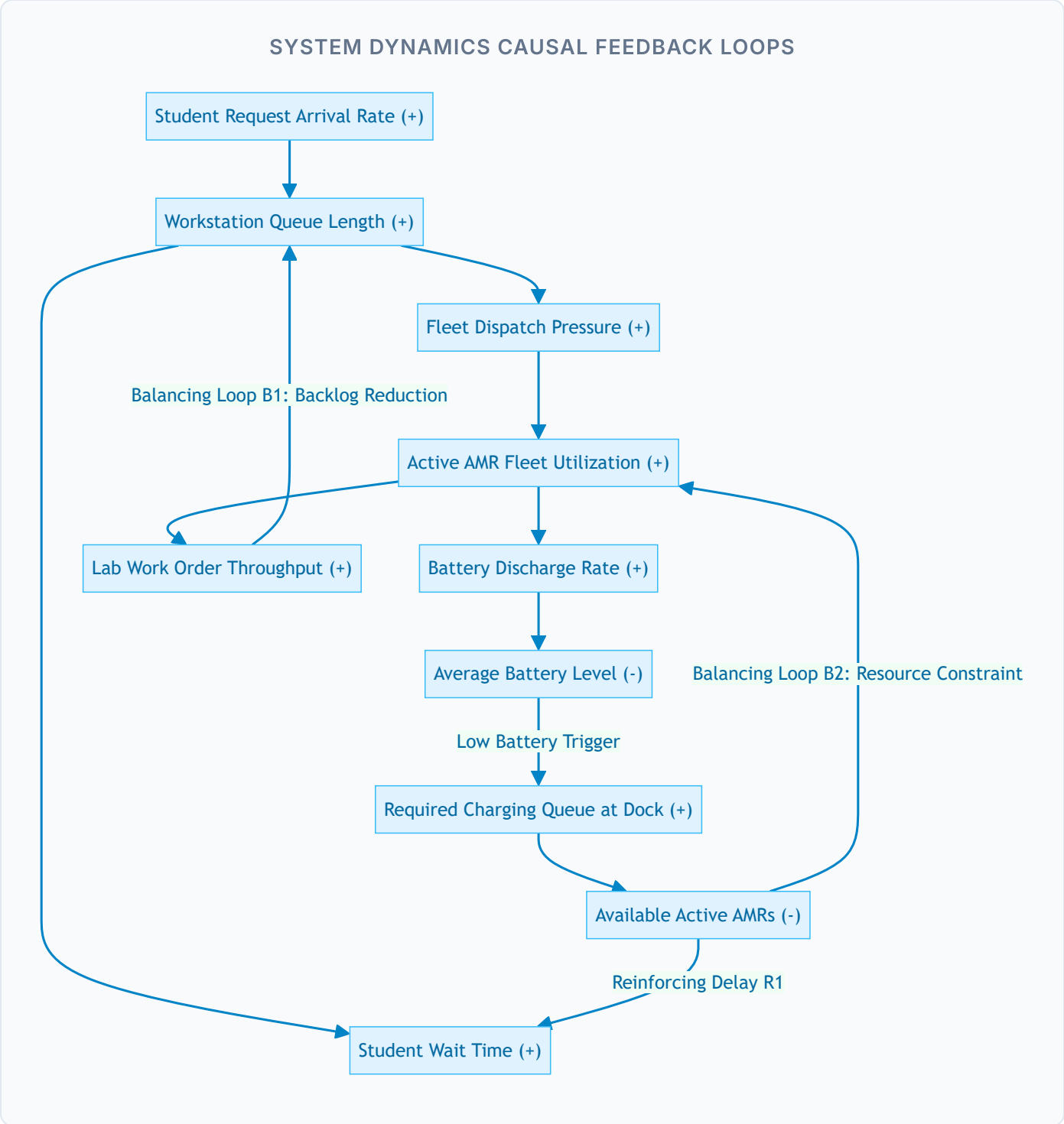
- Execute 5 AnyLogic experiments (Optimization, Monte Carlo, etc.)
- Draft 2–3 page technical summary paper with recommendations
- Assemble 10–12 minute presentation slide deck
- Package final unified single `.zip` submission package

> **Mandatory Individual Deliverable (10% of Grade):** Each member (Jorge, Jordan, Illia, Ricardo) must also independently complete and zip **Module 4: Lesson 1** (Stages 1–3) to include in the final team archive.

3. Conceptual Model Diagrams

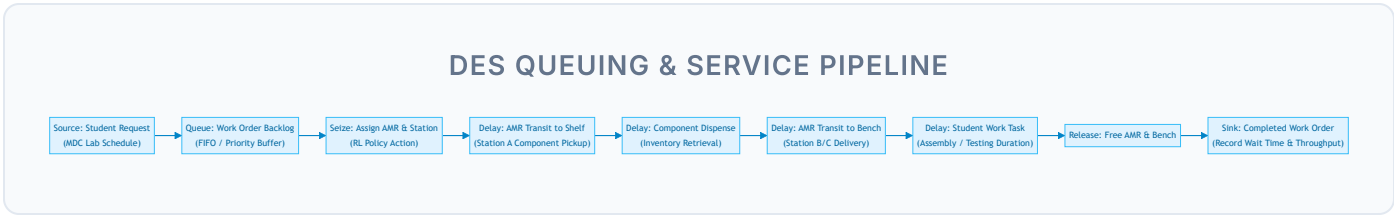
A. Causal-Loop Diagram (CLD)

Models the macro system feedback loops governing student requests, queue backlogs, fleet utilization, and battery depletion:



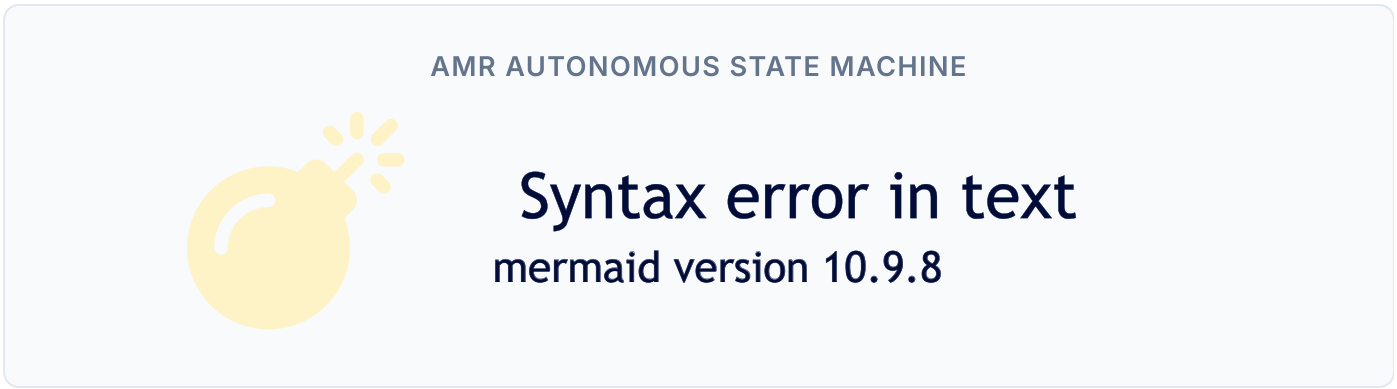
B. Discrete-Event Simulation (DES) Process Flow

AnyLogic queuing architecture modeling component requests, inventory retrieval, and workstation testing:



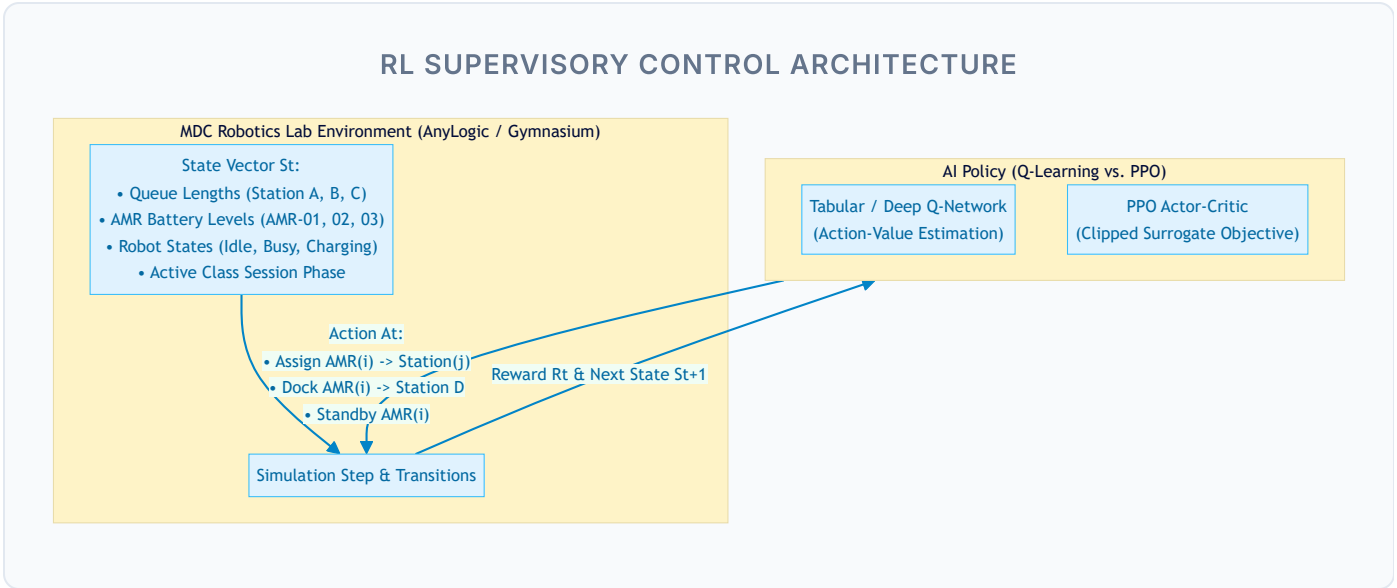
C. Agent-Based Modeling (ABM) AMR Statechart

Individual robot state machine showing task execution, navigation, and automated battery management:



D. Reinforcement Learning (RL) Decision Framework

Markov Decision Process (MDP) comparing **Q-Learning** against **PPO**:



4. Required AnyLogic Simulation Experiments

Experiment	Purpose & Methodology	Target Output
Optimization	Find optimal fleet size (2 vs 3 vs 5 AMRs) and charging pads under lab budget constraints.	Pareto frontier of min wait time vs. capital cost.
Compare Runs	Direct head-to-head comparison between FIFO baseline, Q-Learning, and PPO policies.	Comparative throughput and wait-time bar charts.
Monte Carlo	100+ replications with stochastic student arrival intervals and task durations.	95% confidence interval distributions of student wait times.
Sensitivity Analysis	Vary battery drain rate (aging batteries) and student influx (±30%).	Spider charts identifying tipping points for lab gridlock.
Calibration	Tune simulated robot velocities and service times against physical MDC lab stopwatch data.	Pre- vs. Post-calibration curve alignment.

5. Streamlit Dashboard Architecture (5 Tabs)

Tab	Name	Features & Controls
Tab 1	Control Center & Inputs	Interactive sliders: Student arrival frequency, AMR count, transit speed, policy selector.
Tab 2	Operational KPIs	Real-time metrics: Average wait time, throughput/hr, station utilization heatmaps.
Tab 3	RL Benchmark (Q-Learning vs. PPO)	Episode reward curves, loss progression, policy execution efficiency tables.
Tab 4	Simulation Experiments	Visualizers for Monte Carlo density, sensitivity spider plots, and optimization results.
Tab 5	Executive Report	Non-obvious findings, policy recommendations, and actionable operational strategies for MDC.

6. Final Submission Checklist

- [ ] AnyLogic simulation model file (`.alp`) containing DES + ABM + 3D animation
- [ ] Python RL model training files and Streamlit Dashboard source code
- [ ] Short written summary report (2–3 pages)
- [ ] Professional presentation slide deck (10–12 minutes)
- [ ] Module 4 Lesson 1 zip archives for Jorge A Mendoza
- [ ] Module 4 Lesson 1 zip archives for Jordan Gonzalez
- [ ] Module 4 Lesson 1 zip archives for Illia Shybanov
- [ ] Module 4 Lesson 1 zip archives for Ricardo Molina Gonzalez
- [ ] Group Peer Evaluation grading rubrics completed and signed
- [ ] Single unified zip file assembled for final submission